

MelodyScript: A Domain-Specific Language & Compiler for Algorithmic Music Composition

BCSE307P Compiler Design Lab - Review 1

Presenter: Sanjaay B — Register No: 24BCE2181 | B.Tech CSE, SCOPE, VIT Vellore (2026–27)

Key focus: Hand-crafted multi-pass Java compiler that emits Standard MIDI File (SMF, Format 0) binary output for immediate audible verification.

Topic Selection & Academic Relevance

- Domain: External DSL for algorithmic music composition — concise high-level syntax for musical ideas.
- Structural symmetry: Musical primitives (pitch, duration, repeat) map cleanly to EBNF → AST → symbol tables.
- Practical rigor: Targets Standard MIDI File Format 0 binary spec rather than toy interpreters.
- Audible verification: Compiler output is a .mid file; correctness validated by direct audio playback.

Problem Statement & Significance

Current workflows for programmatic music either (a) manipulate MIDI bytes ad-hoc (error-prone) or (b) rely on runtime libraries with limited static guarantees. These approaches obscure low-level details and delay bug discovery until playback.

MelodyScript provides: compile-time static checks (e.g., MIDI pitch bounds 0-127, positive tempo, valid scoping) and direct binary code generation to the SMF specification, reducing runtime surprises and enabling earlier verification.

Project Objectives & Scope

Formal Grammar

Complete EBNF for musical primitives and control constructs (phrases, repeats, tempo, instrument).

Hand-crafted Front-end

Lexer + LL(1) recursive-descent parser implemented in Java (no parser generators).

Multi-pass Semantics

Scoped symbol table, type-like checks (pitch range, duration positivity), name resolution for phrases.

IR & Codegen

IR flattens repeats; backend writes MThd/MTrk chunks and VLQ-encoded delta-times into SMF Format 0.

Technical Feasibility & Tech Stack

Language & Libraries

- Java (JDK 17+)
- JUnit 5 for unit & regression tests
- javax.sound.midi used only for test assertions & playback verification

Target: Standard MIDI File Spec 1.0 — Format 0 (single multi-event track)

Feasibility

- 100% self-contained — no runtime parser libraries (ANTLR/Lex/Yacc) or external bytecode runtimes
- Deterministic output: binary .mid files reproducible from same source

Compiler Concepts & EBNF Grammar

Mapping of classic compiler stages to MelodyScript modules:

Compiler Stage	MelodyScript Module
Lexical Analysis	Lexer / Tokenizer
Syntax Analysis	Recursive-descent Parser → AST
Semantic Analysis	Semantic Analyzer: pitch bounds, durations, scoping
Symbol Table	Scoped symbol table (phrases, instruments, named motifs)
IR Generation	IR Generator: flattened event list (tick, pitch, duration)
Code Generation	Binary MIDI Writer: MThd/MTrk & VLQ packing

```
<program> ::= (<statement>)*
<statement> ::= "tempo" NUMBER ";" | "instrument" IDENTIFIER ";" | "define" IDENTIFIER "{" <note_sequence> "}" | "play" <target> ";"
<target> ::= IDENTIFIER | "repeat" NUMBER "{" <note_sequence> "}"
<note_sequence> ::= <note> ("," <note>)*
<note> ::= PITCH DURATION
```

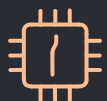
System Architecture & Data Flow



The pipeline is multi-pass:

1. Lexical Analyzer (Tokenizer)
2. Syntax Analyzer (Recursive-Descent Parser -> AST)
3. Semantic Analyzer & Scoped Symbol Table
4. Intermediate Representation (IR Generator)
5. Code Generator (Binary MIDI Byte-Writer)
6. Output Binary MIDI File (.mid)

Innovation & Originality



Direct Binary Codegen

Custom implementation of VLQ encoding and exact MThd/MTrk layout — no intermediary text formats.



Domain-specific Static Analysis

Compile-time checks for music-theoretic constraints (pitch range, tempo positivity, phrase scoping).



Structural Unrolling

Nested repeats and motifs are flattened to linear event lists with precise tick calculations for timing correctness.

Initial Prototype: Sample Input & AST

Sample Source (sample.mel):

```
tempo 120;
instrument piano;

define riff {
  C4 quarter, E4 quarter, G4 quarter, R quarter
}

play riff;
play repeat 2 { A4 eighth, B4 eighth };
```

Generated AST:

```
ProgramNode
├── TempoNode (value: 120)
├── InstrumentNode (name: "piano")
├── PhraseDefNode (identifier: "riff")
│   └── NoteSequenceNode
│       ├── NoteNode (pitch: "C4", duration: "quarter")
│       ├── NoteNode (pitch: "E4", duration: "quarter")
│       ├── NoteNode (pitch: "G4", duration: "quarter")
│       └── NoteNode (pitch: "R", duration: "quarter")
├── PlayNode (target: "riff")
└── PlayNode
    ├── RepeatNode (count: 2)
    │   └── NoteSequenceNode
    │       ├── NoteNode (pitch: "A4", duration: "eighth")
    │       └── NoteNode (pitch: "B4", duration: "eighth")
```

Github Link: <https://github.com/sanjaay617/MelodyScript>